

AI POWERED INTERVIEW PREPARATION PLATFORM

PREPFORGE

Project ID: PRJ071 | **Team ID:** T071 | **Team Name:** TFI

Team GitHub Repository:
github.com/navaneswarreddy007-design/ai-interview-preparation-platform

Student Name	Registration Number	Individual GitHub Link
N. Navaneswar Reddy	99230040382	github.com/navaneswarreddy007-design
N. Maheshwar Reddy	99230040044	github.com/nmaheshwarreddy123-ctrl
P. Teja Vardhan	99230040405	github.com/Teja2201-dev
T. Anjaneya Reddy	99230040437	github.com/ANJANEYAREDDY-hub
A. Hareesh Kumar	99230040076	github.com/attappahareshkumar-a11y

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Project Overview

Preparing for an interview can be challenging because candidates must understand job requirements, present their skills clearly, and practise technical and behavioural questions. PrepForge is a web-based interview preparation platform designed to bring these activities into one guided workflow.

The application accepts a candidate's resume, extracts relevant information, supports interview question generation, provides an answer-practice environment, and presents feedback and session reports. It also includes browser-based voice interaction and a Retrieval-Augmented Generation (RAG) Knowledge Assistant that retrieves relevant material from an indexed interview curriculum knowledge base.

Prepare	Practise	Improve
Upload a resume and use candidate information. Ask the Knowledge Assistant questions grounded in indexed learning resources.	Respond to technical and behavioural questions using text or supported voice input.	Review scores, feedback, interview history, cited knowledge sources and downloadable reports.

Project identity

Item	Details
Project title	AI Powered Interview Preparation Platform
Product name	PrepForge
Application type	Web-based interview preparation platform with RAG Knowledge Assistant
Repository	github.com/navaneswarreddy007-design/ai-interview-preparation-platform
Live deployment	https://ai-interview-preparation-platform-cd5r.onrender.com

Problem Statement

Students and job seekers often find it difficult to prepare systematically for interviews. Preparation may involve separate tools for resume review, question practice, answer evaluation, voice practice, technical concept revision and progress tracking.

Key challenges addressed by the platform:

• Candidates may be unsure which interview questions are relevant to their resume and target role.

• Practising answers without structured feedback makes it difficult to identify areas for improvement.

• Switching between separate tools for resume parsing, question practice and progress review can interrupt preparation.

• Some candidates prefer speaking their answers rather than typing them, so voice-based practice is useful.

• Candidates need a way to review previous sessions and retain a record of their preparation.

• Interview learners need a way to ask technical questions and receive answers linked to relevant curriculum material.

Project need: A single platform that supports resume-informed preparation, interview practice, feedback, session review and source-grounded technical learning in a guided workflow.

Project Objectives

Objective	Description
01 • Resume-based preparation	Extract text and candidate details from supported PDF and DOCX resumes.
02 • Interview question generation	Prepare interview questions using available resume and role-related information.
03 • Answer evaluation	Assess submitted answers and provide scores, strengths, weaknesses and suggestions.
04 • Voice interaction	Support speech-to-text answer entry and text-to-speech playback where browser capabilities allow.
05 • Progress tracking	Store interview sessions and make previous practice history available.
06 • Report generation	Generate downloadable PDF reports containing interview performance information.
07 • Knowledge assistance	Retrieve relevant passages from the indexed interview knowledge base and present grounded answers with source attribution.
08 • Efficient embedding inference	Use ONNX Runtime with an INT8 embedding model to reduce runtime memory requirements compared with the earlier PyTorch-based embedding stack.

The objectives combine the core interview workflow with a technical knowledge assistant. The Knowledge Assistant is a separate learning support capability and does not replace the platform’s interview evaluation workflow.

Proposed Solution

PrepForge combines resume processing, interview practice, performance review and knowledge retrieval in one web application. The workflow is organised around the candidate's preparation session.

Resume preparation path	Interview practice path
1. Candidate uploads a PDF or DOCX resume. 2. Backend extracts available text. 3. Candidate information is used in the preparation workflow.	1. Candidate starts an interview practice session. 2. Questions are presented. 3. Candidate answers using text or voice input.

Evaluation and review path	Knowledge Assistant path
1. Submitted answers are evaluated by the configured evaluation engine. 2. Feedback and scoring are returned. 3. Session information can be reviewed and exported.	1. User submits a technical question. 2. Query embedding is generated using ONNX Runtime. 3. ChromaDB retrieves relevant knowledge chunks. 4. A grounded response is synthesized and source references are displayed.

AI provider and fallback

Gemini and OpenAI provider adapters are present in the codebase. Their use depends on valid runtime configuration and successful provider responses. For the Knowledge Assistant, the verified live ACID test showed "Knowledge Base Synthesis (Rule-Based Grounded Retrieval)"; external LLM inference was not used for that response.

RAG is implemented in the current version. The Knowledge Assistant uses document ingestion, chunking, embeddings, vector retrieval, relevance filtering, source consolidation and grounded response synthesis.

System Architecture

High-level view of the application components

Candidate interface	Frontend API layer	Flask backend	Persistence
React web application Resume upload • interview room • history • Knowledge Assistant	Sends requests and displays backend responses.	Routes, validation, application services and RAG API endpoints.	SQLite session and practice data; ChromaDB vector collection for knowledge retrieval.

Resume processing	Question and evaluation services
PDF/DOCX text extraction and candidate profile preparation.	Question generation, answer scoring and feedback logic.
Speech services	RAG knowledge services
Browser Web Speech API for speech recognition and speech playback, subject to browser support and permissions.	Document parsing → chunking → ONNX embeddings → ChromaDB retrieval → grounded answer synthesis and source citations.
Reporting and deployment	Runtime optimisation
ReportLab PDF export; Docker-based deployment configuration for Render.	ONNX Runtime CPU inference, INT8 embedding model and a single Gunicorn gthread worker for constrained deployment memory.

Application flow: Candidate → React frontend → Flask API → application services → SQLite / file processing / evaluation / RAG retrieval → response to frontend.

Knowledge retrieval flow: User query → tokenizer → ONNX embedding model → query vector → ChromaDB similarity search → topic/relevance filtering → source deduplication → grounded synthesis → cited response.

The RAG Knowledge Assistant is implemented. It is distinct from resume parsing and interview answer scoring; those existing workflows continue to use their respective services.

Technology Stack

Layer	Technology / Component	Purpose
Frontend	React 18.3.1, Vite, Tailwind CSS	Builds the interactive web interface and responsive layouts.
UI components	Lucide icons	Provides interface icons.
Browser voice	Web Speech API / Web Audio API	Supports speech recognition and audio-related interaction in supported browsers.
Backend	Python 3.11, Flask, Flask-CORS	Handles application routes, request processing and frontend communication.
Resume processing	pypdf, python-docx	Extracts text from supported PDF and DOCX resumes.
Database	SQLite	Stores interview sessions, questions, practice drills and related records.
Vector store	ChromaDB	Stores and retrieves indexed knowledge chunks using vector similarity.
Embedding runtime	ONNX Runtime (CPU)	Runs the embedding model without loading PyTorch/SentenceTransformers.
Embedding model	Xenova/all-MiniLM-L6-v2 INT8 ONNX	Produces 384-dimensional query and document embeddings.
Tokenizer and arrays	Hugging Face Tokenizers, NumPy	Tokenizes text and performs mean pooling and L2 normalization.
Knowledge ingestion	TXT, PDF and DOCX parsing; boundary-aware chunking	Converts curriculum files into searchable text chunks with metadata.
AI provider adapters	Gemini and OpenAI adapters	Optional external model integration when configured and available.
Fallback evaluation	Rule-based evaluation and grounded synthesis	Provides deterministic fallback behaviour when external model inference is unavailable.
PDF reporting	ReportLab	Creates downloadable interview reports.
Deployment	Docker, Render	Packages and hosts the web application.

External AI usage depends on valid runtime configuration and successful provider responses. The live ACID Knowledge Assistant test reported rule-based grounded synthesis rather than external LLM inference.

Implementation Details

Frontend	Backend	Data stores
Displays dashboard, resume upload, interview questions, voice/text answer area, evaluation feedback, history, settings and Knowledge Assistant source cards.	Receives frontend requests, validates data, processes resumes, manages sessions and coordinates question, evaluation and RAG services.	SQLite persists interview activity. ChromaDB stores indexed knowledge vectors and chunk metadata.

Main processing sequence

1. Candidate opens PrepForge and starts the preparation workflow.
2. Candidate uploads a supported resume file.
3. Backend extracts resume text and prepares candidate information.
4. The application presents interview questions for practice.
5. Candidate submits an answer through text entry or voice transcription.
6. The evaluation service returns a score and feedback using the configured provider or fallback.
7. Session details are stored and can be reviewed or exported as a PDF report.

Knowledge Assistant processing sequence

1. Knowledge documents are loaded from the knowledge-base directory.
2. TXT, PDF and DOCX content is parsed and divided into boundary-aware chunks.
3. Each chunk is embedded and stored in the ChromaDB collection **prepforge_interview_knowledge**.
4. At query time, the user’s question is embedded with the same ONNX embedding pipeline.
5. ChromaDB returns candidate chunks; topic-aware relevance filtering and deduplication refine the results.
6. The answer is synthesized from retrieved context and displayed with source document and section information.

Core backend modules

Module / component	Responsibility
app.py	Flask application routes and API request handling.
question_generator.py	Interview question and follow-up generation logic.
evaluator.py	Answer evaluation, scoring and feedback logic.
database.py	SQLite database access and persistence.
backend/rag/embeddings.py	ONNX INT8 embedding inference and vector generation.
backend/rag/vector_store.py	Knowledge chunk storage, similarity retrieval, relevance filtering and deduplication.
backend/rag/generator.py	Grounded answer synthesis and source metadata preparation.

RAG Knowledge Assistant

The Knowledge Assistant enables users to ask technical interview and curriculum questions. It retrieves relevant passages from the project's indexed knowledge base and uses those passages to construct a grounded response.

Knowledge base

The indexed knowledge base contains nine source documents covering core interview subjects. The current ChromaDB collection contains 66 indexed chunks.

Knowledge area	Indexed source document
DBMS and SQL	dbms_and_sql_concepts.txt
Operating systems	operating_systems_concepts.txt
Data structures and algorithms	dsa_fundamentals.txt
Computer networks	computer_networks_fundamentals.txt
Python and Java	python_and_java_fundamentals.txt
Machine learning and AI	machine_learning_and_ai.txt
HR and behavioural interviews	hr_and_behavioral_star_method.txt
System design	system_design_interview_handbook.pdf
Distributed systems	distributed_systems_patterns.docx

Retrieval and source quality controls

- Boundary-aware chunking preserves document sections and associated metadata.
- Topic-specific keyword checks are separated from generic domain terms to reduce unrelated matches.
- Candidate chunks are filtered using relevance criteria; the system does not pad results with irrelevant Top-K chunks.
- Chunks from the same document and section are consolidated, while distinct useful paragraphs are preserved.
- Source cards expose content, document name, section title, page information where available and similarity score.
- Source references are deduplicated so displayed citations correspond to retrieved context used for synthesis.

Indexed knowledge base: 9 documents • 66 chunks • 384-dimensional embeddings.

ONNX INT8 Embedding Pipeline

The earlier embedding implementation used the PyTorch/SentenceTransformers stack. To reduce memory pressure on the Render Free instance, the embedding path was migrated to ONNX Runtime with an INT8-quantized MiniLM model.

Runtime design

Stage	Implementation
Model	Xenova/all-MiniLM-L6-v2 INT8 ONNX model.
Inference runtime	ONNX Runtime CPU execution; inference thread count constrained for a small instance.
Tokenization	Standalone Hugging Face Tokenizers implementation.
Pooling	Attention-mask-aware mean pooling over token embeddings.
Normalization	L2 normalization of the resulting embedding vector.
Output	384-dimensional vector used for ChromaDB storage and query retrieval.
Input handling	Tokenizer padding and truncation configured to a maximum sequence length of 256 tokens.

Deployment optimisation

The Docker build downloads the model and tokenizer into the image cache and performs knowledge-base indexing during image build. This avoids relying on a large model download and full re-index operation during normal application startup. Runtime model loading is lazy, and the Render service uses one Gunicorn worker with the gthread worker class.

Compatibility and verification

The ONNX embedding manager preserves the embedding interface used by the RAG pipeline. The reported local comparison showed cosine similarity of approximately 1.0000 for ONNX FP32 and 0.9968 for INT8 against the reference embeddings. Local verification reported 384-dimensional embeddings and successful retrieval for the five regression topics. These are local test results; they are not a measurement of production Render memory usage.

The ONNX INT8 migration is intended to reduce runtime memory consumption. Actual Render RAM and CPU metrics are not visible on the Free plan, so production memory usage has not been independently measured.

Interview Workflow and RAG API

Interview practice workflow

Step	User / system activity
01 • Open PrepForge	Candidate enters the web application.
02 • Upload resume	Candidate provides a supported PDF or DOCX resume.
03 • Prepare questions	Questions use available candidate and role information.
04 • Answer questions	Candidate types an answer or uses speech recognition where supported.
05 • Evaluate answer	Backend evaluates the submitted response and prepares feedback.
06 • Review progress	Candidate reviews session results, history and PDF report.
07 • Ask the Knowledge Assistant	Candidate submits a technical question and receives a response grounded in retrieved curriculum material with source attribution.

RAG API endpoints

Method and endpoint	Purpose
GET /api/rag/status	Returns Knowledge Assistant status and indexing information.
GET /api/rag/documents	Lists the available indexed knowledge documents.
POST /api/rag/retrieve	Retrieves relevant knowledge chunks for a query.
POST /api/rag/query	Runs retrieval and returns a grounded answer with source information.
POST /api/rag/ingest	Triggers knowledge-base ingestion and indexing.

The ingestion endpoint is an administrative operation and may require additional memory while processing documents. The normal deployed workflow uses build-time indexing to avoid routine startup re-indexing.

Testing and Validation

The project has been checked through automated backend tests, RAG regression tests, local HTTP verification and a frontend production build. The live deployment was also opened and a Knowledge Assistant ACID query returned a grounded response with source attribution.

Test area	Validation focus	Reported result
RAG pipeline regression	Automated RAG pipeline test suite	18 / 18 PASS
Core interview audit	Interview and scoring audit suite	5 / 5 PASS
RAG topic regression	ACID, SQL joins, Coffman deadlock conditions, Python GIL, CAP/PACELC	5 topics locally verified
ACID retrieval	Correct ACID section, bank transfer example, no unrelated SQL join source in local regression	PASS
SQL joins retrieval	Join strategies section retrieved for SQL join query	PASS
Deadlock retrieval	Deadlock section and four Coffman conditions retrieved	PASS
Python GIL retrieval	Python internals section retrieved	PASS
CAP/PACELC retrieval	Distributed systems section retrieved	PASS
Source display	Non-empty chunk content and deduplicated source cards in local verification	PASS
Frontend production build	Vite production build	PASS
Live RAG query	ACID question returned a grounded response and source excerpt on Render	PASS
Render deployment	Commit 2eedf71 shown as Live	PASS
Render memory metrics	Application RAM/CPU metrics on Free plan	NOT AVAILABLE
Physical microphone and speaker	Actual browser permission and audio playback	MANUAL VERIFICATION

The local regression results and live deployment check are separate forms of verification. Render’s Free plan does not expose application memory/CPU metrics, so the ONNX deployment’s exact production memory footprint is not reported here. A successful live query confirms functionality for that test, not complete load or long-duration stability.

Conclusion and Future Scope

Conclusion	Future enhancements
PrepForge brings resume-informed interview preparation, question practice, voice interaction, answer feedback, session history, PDF reporting and a source-grounded Knowledge Assistant into one web-based platform. Its modular frontend and backend structure supports a guided practice workflow. The RAG pipeline uses a ChromaDB knowledge collection and ONNX INT8 embeddings for retrieval.	Further work can improve semantic answer evaluation, validate external AI provider connectivity in production, expand browser voice compatibility, strengthen retrieval evaluation with a larger benchmark set, add richer analytics, monitor resource use under load and conduct broader real-user testing.

THANK YOU

Questions & Discussion

PrepForge • AI Powered Interview Preparation Platform

Project Trainer / Guide: Mr. SHIRWIN PRINCE I

Project references

GitHub repository: https://github.com/navaneswarreddy007-design/ai-interview-preparation-platform
Live application: https://ai-interview-preparation-platform-cd5r.onrender.com
Deployment commit: 2eedf71 — Migrate PrepForge RAG to ONNX INT8 for low memory

Documentation note: This report describes implementation and verification information available at preparation time. External AI provider use, physical voice hardware behaviour and production memory measurements remain subject to separate verification.